Minimal Viable Product Documentation - Prototype Phase (Updated with PHC Dashboard)

1. Problem

ASHA workers need to send SMS notifications to patients/hospitals even without internet connectivity. Current solutions rely on web apps that don't work in the field with intermittent connectivity. The prototype must provide an offline-first mobile experience where messages are stored locally and sent automatically when network returns, without authentication overhead for this phase. Additionally, PHC supervisors need a dashboard to view referral requests from ASHA workers.

2. Architecture

Three-Tier System:

- **Flutter Mobile App** (ASHA worker's handset) - offline-first, local queue, connectivity watcher
- **Node.js/Express Backend** - REST API, Redis queue for SMS, PostgreSQL persistence, referral request handling
- **PostgreSQL Database** - stores users, items, SMS queue status, referral requests; Twilio delivery receipts

Data Flow:

Flutter App (local queue)
  !“
Backend REST API (/api/sms/queue, /api/referrals)
  !“
Redis Message Queue / PostgreSQL
  !“
Twilio SMS Sender
  !“
PostgreSQL (status updates)
  !“
Flutter App (polling for status)

  !“
[PHC Dashboard]
  !“
View Referrals

3. API Declarations (No Auth - Prototype Phase)

SMS Endpoints:

Method	Endpoint	Description
`POST`	`/api/sms/send`	Send SMS immediately (no queue)
`GET`	`/api/sms/queue`	List pending SMS messages (status: pending/sent/failed)
`POST`	`/api/sms/queue`	Add SMS to offline queue
`GET`	`/api/health`	Health check endpoint

Referral Endpoints:

Method	Endpoint	Description
`POST`	`/api/referrals`	Create new referral request (from ASHA SMS)
`GET`	`/api/referrals`	List all referral requests with pagination/filters
`GET`	`/api/referrals/:id`	Get single referral details
`PUT`	`/api/referrals/:id`	Update referral status (pending/accepted/rejected)

Request/Response Models:

SMS Queue POST request:

{

```
  "to": "+919876543210",
  "body": "Your appointment is confirmed for tomorrow at 10 AM."
}
```


SMS Queue response:


```
{
  "id": "msg_abc123",
  "to": "+919876543210",
  "body": "Your appointment is confirmed...",
  "status": "pending",
  "createdAt": "2026-01-15T10:30:00Z",
  "sentAt": null
}
```


Referral CREATE request:


```
{
  "from": "ASHA Worker",
  "to": "PHC Hospital",
  "type": "general",
  "message": "Patient needs maternity checkup",
  "priority": "normal"
}
```


Referral response:


```
{
  "id": "ref_abc123",
  "from": "ASHA Worker",
  "to": "PHC Hospital",
  "type": "general",
  "message": "Patient needs maternity checkup",
  "status": "pending",
  "createdAt": "2026-01-15T10:30:00Z",
  "updatedAt": "2026-01-15T10:30:00Z"
}
```


Health check:


```
{
  "status": "ok",
  "timestamp": "2026-01-15T10:30:00Z",
  "version": "v1.0.0"
}
```


4. Offline SMS Feature - Minimal Implementation

Frontend (Flutter):

- `ConnectivityPlus` package watches network state
- Local storage (IndexedDB via `idb` or `SQFlite`) stores SMS queue
- UI: Compose form with phone number + message body
- On send tap: Save to local storage + try sending via backend
- On app startup: Sync local queue to backend (`POST /api/sms/queue`)
- Poll `GET /api/sms/queue` every 30s to update UI status

Backend (Node/Express):

- `POST /api/sms/queue` - receives SMS, saves to PostgreSQL with status='pending', adds to Redis queue
- Worker process: Redis loop pops messages, sends via Twilio, updates DB status to 'sent' or 'failed'
- `GET /api/sms/queue` - returns messages filtered by status, paginated
- `POST /api/referrals` - creates referral request from SMS context, stores in DB with status='pending'
- `GET /api/referrals` - returns all referrals with optional filters (status, type, priority)

SMS Send Flow:

1. ASHA types message !' taps send
2. App saves to local storage with status='pending'
3. App attempts `POST /api/sms/queue` !' backend saves to DB, adds to Redis
4. Backend worker picks from Redis !' sends via Twilio
5. Twilio responds with message SID !' backend updates DB status='sent', records sentAt
6. If Twilio fails !' backend updates status='failed', stores error code
7. App polls status !' shows delivered/failed status
8. Backend automatically creates referral request when SMS is sent (optional)

5. PHC Dashboard - Referral Requests

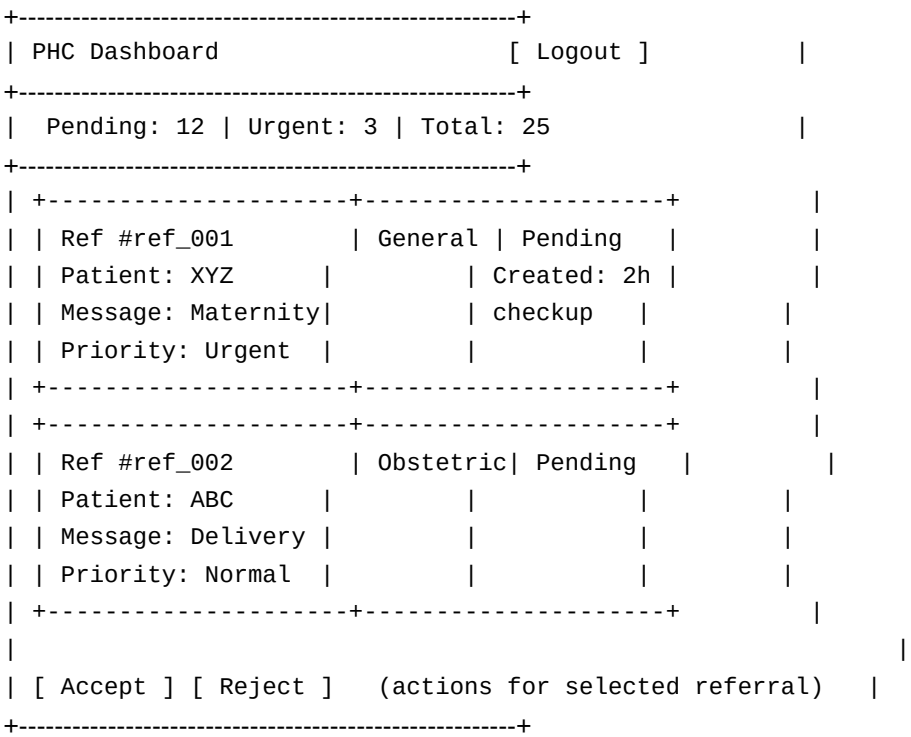
Purpose:

Primary Health Center supervisors view and manage referral requests coming from ASHA workers.

Dashboard Features:

1. **Referral List** - Table showing all referrals with columns: ID, From (ASHA), Type, Message, Status, Created Date, Priority
2. **Filtering** - Filter by status (pending/accepted/rejected), type (general/obstetric/pediatric), priority (normal/urgent/emergency)
3. **Referral Detail View** - Clickable row shows full message, sender contact info, and action buttons
4. **Status Actions** - "Accept" and "Reject" buttons to update referral status
5. **Count Badges** - Pending referrals count, urgent referrals count

UI Layout (Mobile-Responsive):



6. Referral Request Flow

How Referrals Are Created:

1. ASHA sends SMS to patient/hospital via the mobile app
2. Backend receives SMS at `POST /api/sms/queue`
3. Backend worker process creates a referral request automatically
4. Referral stored in PostgreSQL with status='pending'
5. PHC Dashboard polls `GET /api/referrals` to show new referrals
6. Supervisor views referral and clicks "Accept" or "Reject"
7. `PUT /api/referrals/:id` updates status to 'accepted' or 'rejected'

Referral Model (Prisma):

```
model Referral {
  id        String   @id @default(uuid())
  from      String   // "ASHA Worker" or sender identifier
  to        String   // "PHC Hospital" or recipient
  type      String   // general | obstetric | pediatric
  priority  String   // normal | urgent | emergency
  message   String   // description of the referral reason
  status    String   @default('pending') // pending | accepted | rejected
  createdAt DateTime @default(now())
  updatedAt DateTime @default(now())
}
```

7. Other MVP Features

Core Features (Flutter App):

1. **SMS Compose** - Input field for phone number, message body, send button
2. **Connectivity-aware** - auto-saves draft when network drops, queues for later
3. **Status Display** - shows sent/failed count from backend polling
4. **Simple Dashboard** - total messages sent today/this session

Backend Features:

1. **Health endpoint** - for monitoring
2. **SMS persistence** - PostgreSQL table with indexes on status + createdAt
3. **Redis queue** - lightweight message broker for outgoing SMS
4. **Referral creation** - automatic referral generation from SMS sends
5. **Error logging** - Twilio error codes stored for debugging

8. Out of Scope (Prototype Phase)

- ' User authentication/authorization (skipped)
- ' Role-based access control
- ' Email verification flows
- ' Admin dashboard with complex analytics
- ' Push notifications (FCM/APNs)
- ' Image/upload handling
- ' Multi-language support
- ' Complex filtering/search beyond basic status/type filters
- ' Production Twilio pricing (using trial/sandbox)

9. Success Metrics for Prototype

- [ ] ASHA can compose and send SMS without internet connection
- [ ] Message is stored locally and sent automatically when network restores

- [ ] Backend receives and logs delivery status (sent/failed)

- [ ] App UI updates to show message status
- [ ] PHC Dashboard displays referral requests from ASHA workers
- [ ] Supervisor can filter referrals by status, type, priority
- [ ] Supervisor can accept/reject referral requests
- [ ] Backend health endpoint responds `{"status":"ok"}`
- [ ] No crashes on app background/foreground transitions
- [ ] SMS queue persists across app restarts

10. Open Questions (Require User Decision)

1. **Local storage preference:** IndexedDB (`idb`) vs SQLite (`sqflite`)
2. **Backend hosting:** Railway free tier vs Render free tier
3. **Twilio sandbox vs live:** Use trial account numbers only vs dedicated virtual number
4. **Polling interval:** 15s vs 30s vs 60s battery impact vs latency tradeoff
5. **Auto-referral trigger:** Create referral automatically on every SMS send vs manual ASHA indication
6. **Referral filtering on dashboard:** Show all vs show only pending vs show only urgent

11. PHC Dashboard Tech Details

Backend Endpoints for Dashboard:

- `GET /api/referrals?status=pending&type=obstetric&priority=urgent` - filtered list
- `GET /api/referrals/count?status=pending` - count badges
- `PUT /api/referrals/:id` - update status with audit trail

Database Indexes (for performance):

- `index: { status: 1, createdAt: -1 }` - for listing pending referrals sorted by date
- `index: { type: 1, status: 1 }` - for filtering by type and status
- `index: { priority: 1 }` - for priority-based filtering

USP: "ASHA workers can send SMS to patients even without internet connectivity - messages are saved locally on the device and sent automatically when network returns, enabling reliable communication in rural areas with intermittent connectivity. PHC supervisors gain real-time visibility into referral requests from the field without requiring workers to have constant internet access."

This documentation represents the minimal viable product for the prototype phase, incorporating the PHC dashboard with referral request tracking. Authentication, advanced features, and production polish will be addressed in subsequent phases. The Flutter mobile app serves as the true endpoint in the ASHA worker's hand, with the backend and SMS infrastructure supporting offline functionality, while the PHC dashboard provides the supervisory view of field referrals.